

AI 500 – Foundations of AI

Written Assignment 1

Deadline: 6th December 2025, Saturday

Name: _____

Roll Number: _____

Instructions

- This is an **individual** assignment. You are expected to solve the questions on your own and not seek help from other students. Please reach out to the TAs if you have any questions regarding the assignment. Plagiarism will not be tolerated.
- You are supposed to **print** the assignment and attempt the questions in the provided space. It is preferred that you present your solutions in the given boxes. Two extra pages are given at the end for more work if needed. You may use a pencil to present your solutions but please ensure readability. **Do not attach more pages.**
- A hard copy of the assignment should be submitted in class before the start of the lecture on **6th December 2025, Saturday**.
- The late submission policy **does not apply** for this assignment. Please note that there will be **no extensions**, and the stated deadline is final.
- All questions are **compulsory**. **Total Marks: 65.**

Best of Luck!

Problem 1: Computational Graphs & Gradient Accumulation [10 points]

Consider the following function which mimics a single neuron with a “skip connection” (a structure where a value is fed into an activation function and also bypasses it):

$$L = \sigma(z) + z^2$$

Where the intermediate value z is a linear combination of input x , weight w , and bias b :

$$z = w \cdot x + b$$

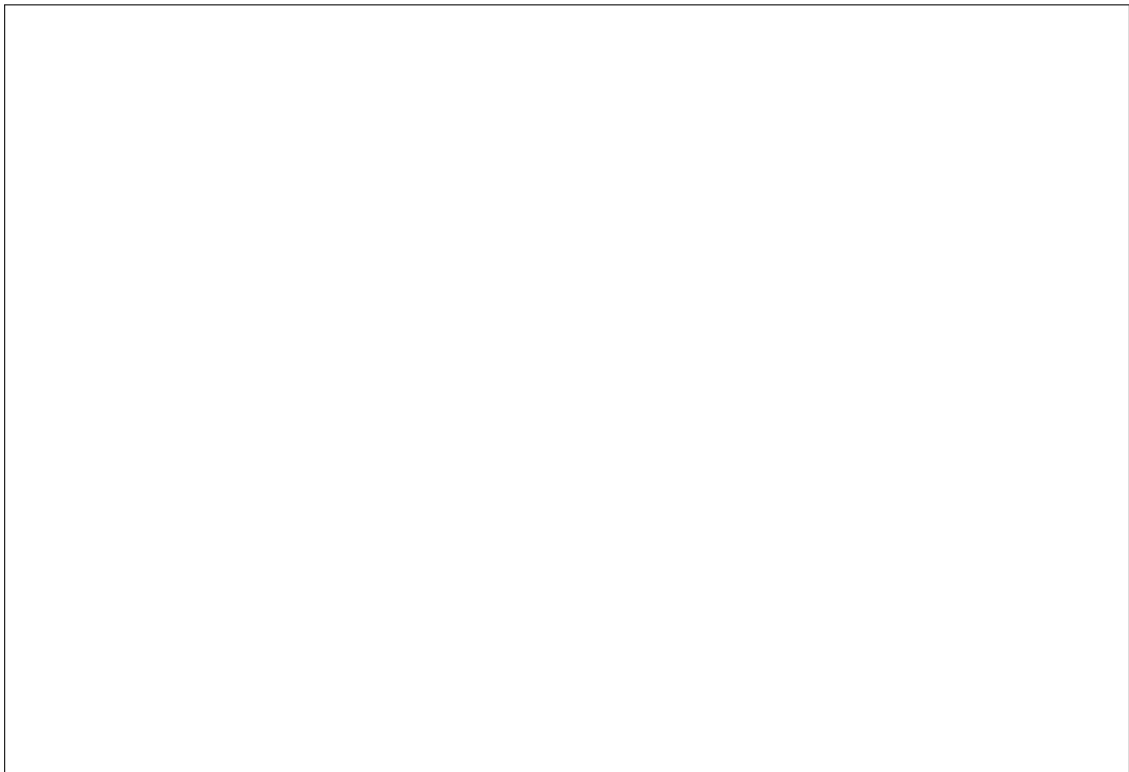
and $\sigma(\cdot)$ is the standard sigmoid function: $\sigma(u) = \frac{1}{1+e^{-u}}$.

Given inputs: $x = 2$, $w = 0.5$, $b = -1$.

Questions

1. [3 Points] **Draw the Computational Graph.**

Decompose the function into elementary operations (addition, multiplication, sigmoid, square). **Crucial Constraint:** You must compute the node z once and show two outgoing branches from it (one to the sigmoid path, one to the square path). Label all intermediate nodes.



2. [3 Points] **Forward Pass.**

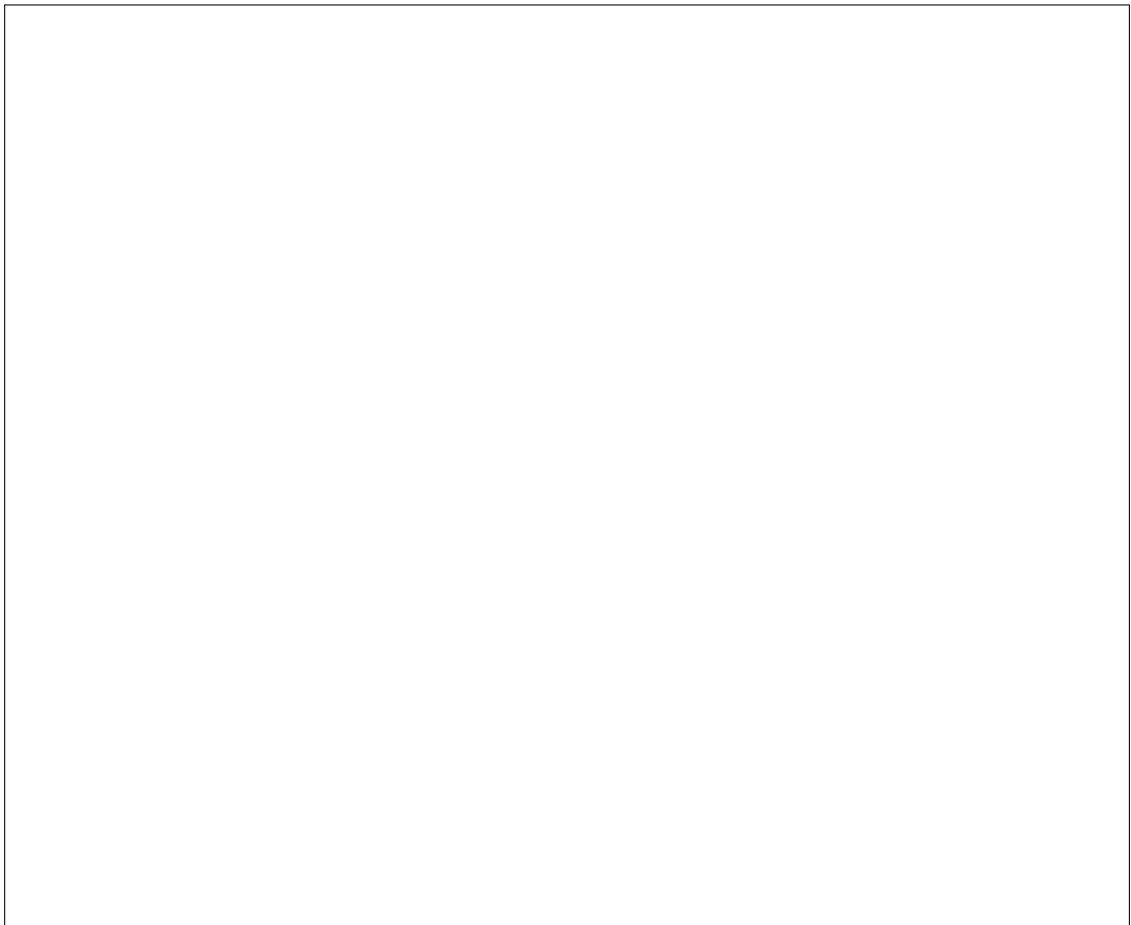
Compute the numerical value of every intermediate node (including z) and the final output L . Show your work.



3. [4 Points] **Backward Pass.**

Compute the gradient $\frac{\partial L}{\partial w}$.

Hint: Since w affects L through two different paths (the sigmoid path and the square path), explicitly show how the gradients from these two branches sum up at node z .



Problem 2: Sparse Feedforward Neural Network with Backpropagation [20 points]

Scenario

A data scientist at a medical diagnostics company is developing a neural network to predict patient risk levels for three different conditions (low blood pressure, normal status, high blood pressure) based on two physiological measurements: resting heart rate and blood oxygen saturation.

Initially, they implemented a fully connected feedforward neural network with 4 hidden neurons. However, after consulting with medical experts, they learned that certain physiological relationships don't exist in practice; for example, resting heart rate alone doesn't directly influence certain intermediate metabolic states, and some metabolic pathways don't affect all three output conditions equally.

To incorporate this domain knowledge and improve model interpretability for clinical use, the data scientist redesigned the network architecture to be **sparse (non-fully connected)**, where connections only exist between neurons that represent physiologically meaningful relationships.

Your task is to analyze this sparse network architecture by performing forward propagation, computing the error, and then using backpropagation to update specific weights based on a single training example.

Consider a **sparse** feedforward neural network with the following architecture:

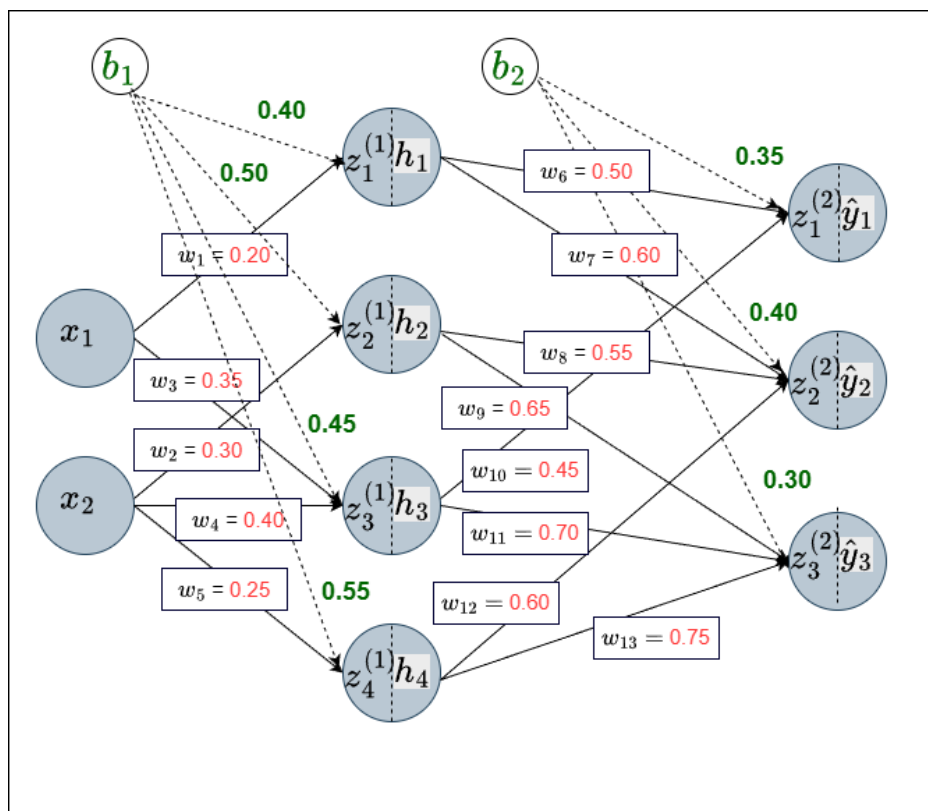


Figure 1: Sparse FFNN Architecture

- **Input layer:** 2 neurons (x_1, x_2)
- **Hidden layer:** 4 neurons (h_1, h_2, h_3, h_4)
- **Output layer:** 3 neurons ($\hat{y}_1, \hat{y}_2, \hat{y}_3$)

All neurons use the **sigmoid activation function**

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Network Connectivity Structure

Input to Hidden Layer Connections:

- x_1 connects to: h_1, h_3
- x_2 connects to: h_2, h_3, h_4

Hidden to Output Layer Connections:

- h_1 connects to: $\hat{y}_1, \hat{y}_2$
- h_2 connects to: $\hat{y}_2, \hat{y}_3$
- h_3 connects to: $\hat{y}_1, \hat{y}_3$
- h_4 connects to: $\hat{y}_2, \hat{y}_3$

Network parameters (initialization)

Input vector:

$$\mathbf{x} = \begin{bmatrix} 0.15 \\ 0.25 \end{bmatrix}$$

Biases (hidden):

$$\mathbf{b}^{(1)} = \begin{bmatrix} 0.40 \\ 0.50 \\ 0.45 \\ 0.55 \end{bmatrix}$$

Biases (output):

$$\mathbf{b}^{(2)} = \begin{bmatrix} 0.35 \\ 0.40 \\ 0.30 \end{bmatrix}$$

Target output:

$$\mathbf{y} = \begin{bmatrix} 0.10 \\ 0.80 \\ 0.90 \end{bmatrix}$$

Learning rate:

$$\eta = 0.5$$

Weight connections:

- $w_1 = 0.20$ connects $x_1 \rightarrow h_1$
- $w_2 = 0.30$ connects $x_2 \rightarrow h_2$
- $w_3 = 0.35$ connects $x_1 \rightarrow h_3$
- $w_4 = 0.40$ connects $x_2 \rightarrow h_3$

- $w_5 = 0.25$ connects $x_2 \rightarrow h_4$
- $w_6 = 0.50$ connects $h_1 \rightarrow \hat{y}_1$
- $w_7 = 0.60$ connects $h_1 \rightarrow \hat{y}_2$
- $w_8 = 0.55$ connects $h_2 \rightarrow \hat{y}_2$
- $w_9 = 0.65$ connects $h_2 \rightarrow \hat{y}_3$
- $w_{10} = 0.45$ connects $h_3 \rightarrow \hat{y}_1$
- $w_{11} = 0.70$ connects $h_3 \rightarrow \hat{y}_3$
- $w_{12} = 0.60$ connects $h_4 \rightarrow \hat{y}_2$
- $w_{13} = 0.75$ connects $h_4 \rightarrow \hat{y}_3$

Question 2.1: Forward Propagation [8 points]

Perform a complete forward pass through the network, respecting the sparse connectivity pattern.

- (a) **[3 points]** Compute the net input and activation for each neuron in the hidden layer. Note that each hidden neuron only receives inputs from specific input neurons according to the connectivity structure. Show all calculations.

- (b) **[3 points]** Compute the net input and activation for each neuron in the output layer. Note that each output neuron only receives inputs from specific hidden neurons according to the connectivity structure. Show all calculations.

- (c) **[2 points]** Calculate the total error using the Mean Squared Error (MSE) loss function:

$$E_{total} = \frac{1}{2} \sum_{k=1}^3 (y_k - \hat{y}_k)^2.$$

Question 2.2: Backward Propagation - Output Layer Weight [5 points]

Consider the weight w_{10} that connects the third hidden neuron (h_3) to the first output neuron (o_1). Note that the derivative of the sigmoid function is: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

- (a) **[4 points]** Using the chain rule, derive and calculate the gradient $\frac{\partial E_{total}}{\partial w_{10}}$. Show each partial derivative in the chain explicitly.

- (b) [1 points] Compute the updated value w_{10}^{new} using gradient descent with learning rate η .

Question 2.3: Backward Propagation - Hidden Layer Weight [7 points]

Consider the weight w_3 that connects the first input neuron (i_1) to the third hidden neuron (h_3). Note that the derivative of the sigmoid function is: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

- (a) [6 points] Using the chain rule, derive and calculate the gradient $\frac{\partial E_{\text{total}}}{\partial w_3}$. Note that h_3 connects to output neurons o_1 and o_3 only. Remember that this weight affects multiple output neurons through the hidden layer. Show all intermediate

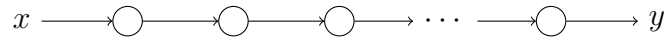
steps including:

- The contribution from each connected output neuron
- The gradient with respect to the hidden neuron activation
- The complete chain rule application

- (b) [**1 points**] Compute the updated value w_3^{new} using gradient descent with learning rate η .

Problem 3: Gradient magnitudes in a chain-like network [6 points]

Suppose we are training the chain-like neural network shown below. Activation functions are logistic (sigmoid) $\sigma(z) = \frac{1}{1+e^{-z}}$. All weights are initially set to 1 and all biases to -0.5 . Giving input $x = 0.5$ causes all unit outputs to be 0.5.



Given a single training example ($x = 0.5$, $y = 1$), what can you say about the order of magnitude of the gradient updates for weights in this network?

Provide both a **qualitative comparison** (how updates for weights near the output compare to those far from the output) and a **numerical derivation** using the chain rule to support your answer. Compute specific values for at least the first few layers and explain the observed pattern (try to characterize the pattern as well, is there a specific name for this problem?)

Problem 4: The Importance of Initialization [4 points]

Refer back to the Feedforward Neural Network architecture from Problem 2.

Suppose that instead of the distinct parameters provided, you initialized **every single weight** in the network to the same constant value (e.g., $w = 0.5$) and all biases to 0.

1. [2 Points] What would be the relationship between the activation values of the hidden neurons (h_1, h_2, h_3) during the forward pass? Explain why.

2. [2 Points] How would this initialization affect the learning process during back-propagation? Specifically, would the hidden neurons simplify or differentiate as training progresses?

Problem 5: Convolution Operations [25 points]

Convolutional Neural Networks (CNNs) have revolutionized computer vision by using convolution operations to detect patterns in data. Unlike fully connected layers where every neuron connects to every input, convolutional layers apply small filters (kernels) that slide across the input, detecting local patterns efficiently.

Important Note on Roll Numbers

For Questions 5.1 and 5.2, you will use digits from your roll number to create personalized input matrices. **Please ensure your roll number is written clearly on the first page.**

Question 5.1: 1D Convolution [4 points]

Consider a 1D convolution operation commonly used in time-series analysis or audio processing.

Setup: Use your 8-digit roll number as the input. For example, if your roll number is 26100129, your input is:

$$\mathbf{x} = [2, 6, 1, 0, 0, 1, 2, 9]$$

Apply the following filter (kernel):

$$\mathbf{f} = [0, 3, 9]$$

Parameters: No padding, stride = 1

- (a) [1 point] What will be the length of the output after convolution? Use the formula:

$$\text{Output length} = \left\lfloor \frac{L_{in} - L_{filter}}{stride} \right\rfloor + 1$$

where L_{in} is input length and L_{filter} is filter length.

- (b) [3 points] Compute the complete convolution output by sliding the filter across the input. Provide the complete output vector.

Question 5.2: 2D Convolution with Padding [6 points]

2D convolutions are the foundation of image processing in CNNs. Each filter detects specific features like edges, corners, or textures.

Setup: Use the **last four digits** of your roll number to create a 2×2 filter. For example, if your roll number is 26100129, use digits 0, 1, 2, 9:

$$\mathbf{F} = \begin{bmatrix} 0 & 1 \\ 2 & 9 \end{bmatrix}$$

Given the input matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Parameters: Zero padding = 1 (on all sides), stride = 1

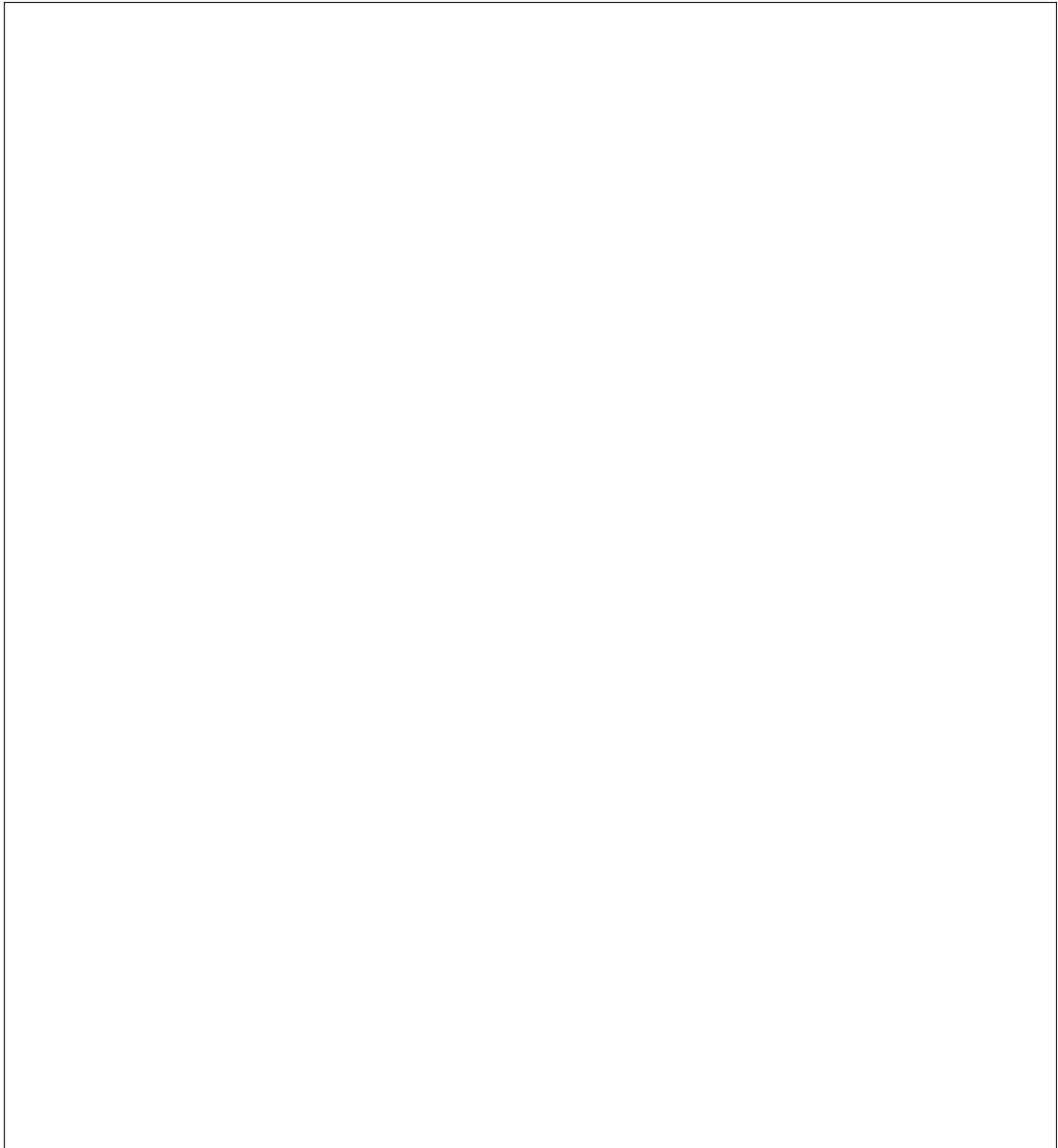
- (a) [1 point] After applying zero padding of 1 to all sides of $\mathbf{X}$, what is the dimension of the padded input? Write out the complete padded matrix.

- (b) [1 point] What will be the dimensions of the output after convolution? Use the formula:

$$\text{Output size} = \left\lfloor \frac{N + 2P - F}{S} \right\rfloor + 1$$

where N is input dimension, P is padding, F is filter size, and S is stride.

- (c) [4 points] Compute the complete convolution output. Show detailed calculations, and provide the complete output matrix.



Question 5.3: Backpropagation Through Convolution - A Guided Proof [15 points]

Goal

In this problem, you will **prove** that backpropagation through a convolutional layer is mathematically equivalent to performing convolution operations. This fundamental insight enables efficient implementation of CNNs and explains why the same hardware optimizations used for forward convolution can be reused during backpropagation.

Setup: Forward Pass

Consider a 2D convolution with no padding and stride=1:

$$\text{Input (from previous layer): } \mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \quad \text{Filter: } \mathbf{F} = \begin{bmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \end{bmatrix} \quad \text{Output:}$$

$$\mathbf{O} = \begin{bmatrix} o_{11} & o_{12} \\ o_{21} & o_{22} \end{bmatrix}$$

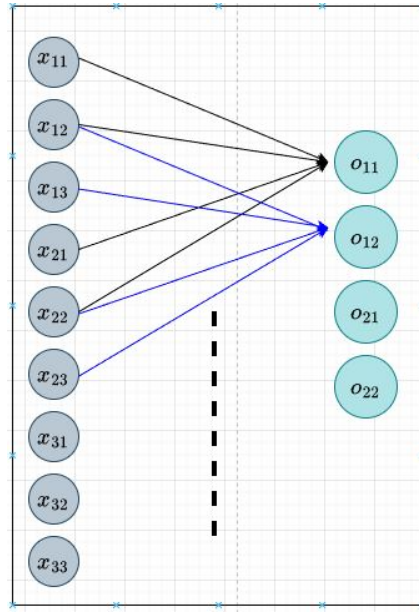


Figure 2: Concept Diagram for Q5

Given: $o_{11} = f_{11}x_{11} + f_{12}x_{12} + f_{21}x_{21} + f_{22}x_{22}$

- (a) **[2 points]** Write explicit expressions for o_{12} , o_{21} , and o_{22} in terms of f_{ij} and x_{ij} .

- (b) **[3 points]** Gradient w.r.t. filter weights:

We receive upstream gradient:

$$\frac{\partial L}{\partial \mathbf{O}} = \begin{bmatrix} \frac{\partial L}{\partial o_{11}} & \frac{\partial L}{\partial o_{12}} \\ \frac{\partial L}{\partial o_{21}} & \frac{\partial L}{\partial o_{22}} \end{bmatrix}$$

We need: $\frac{\partial L}{\partial \mathbf{F}} = \begin{bmatrix} \frac{\partial L}{\partial f_{11}} & \frac{\partial L}{\partial f_{12}} \\ \frac{\partial L}{\partial f_{21}} & \frac{\partial L}{\partial f_{22}} \end{bmatrix}$

Guided derivation for $\frac{\partial L}{\partial f_{11}}$:

By the chain rule: $\frac{\partial L}{\partial f_{11}} = \sum_{i,j} \frac{\partial L}{\partial o_{ij}} \cdot \frac{\partial o_{ij}}{\partial f_{11}}$

From your forward pass equations, f_{11} appears in: $o_{11}, o_{12}, o_{21}, o_{22}$ (all outputs!)

Computing each partial:

$$\frac{\partial o_{11}}{\partial f_{11}} = x_{11} \quad (\text{coefficient of } f_{11} \text{ in } o_{11})$$

$$\frac{\partial o_{12}}{\partial f_{11}} = \text{_____} \quad (\text{fill in})$$

$$\frac{\partial o_{21}}{\partial f_{11}} = \text{_____} \quad (\text{fill in})$$

$$\frac{\partial o_{22}}{\partial f_{11}} = \text{_____} \quad (\text{fill in})$$

Therefore:

$$\frac{\partial L}{\partial f_{11}} = \frac{\partial L}{\partial o_{11}} \cdot x_{11} + \frac{\partial L}{\partial o_{12}} \cdot \text{_____} + \frac{\partial L}{\partial o_{21}} \cdot \text{_____} + \frac{\partial L}{\partial o_{22}} \cdot \text{_____}$$

Now derive: $\frac{\partial L}{\partial f_{12}}, \frac{\partial L}{\partial f_{21}}, \frac{\partial L}{\partial f_{22}}$ using the same method.

- (c) [2 point] Write your four gradient expressions as the matrix $\frac{\partial L}{\partial \mathbf{F}}$. This is equivalent to convolving which two matrices? (State: $\frac{\partial L}{\partial \mathbf{F}} = \text{conv2d}(\text{____}, \text{____})$)

- (d) [3 points] **Gradient w.r.t. input (to propagate backward):**

Compute $\frac{\partial L}{\partial x_{22}}$ (center element):

- Which outputs depend on x_{22} ? -----
- For each: $\frac{\partial o_{ij}}{\partial x_{22}} = \text{-----}$ (which filter weight?)
- Complete: $\frac{\partial L}{\partial x_{22}} = \text{-----}$

Compute $\frac{\partial L}{\partial x_{11}}$ (corner element):

- Which outputs depend on x_{11} ? -----
- Complete: $\frac{\partial L}{\partial x_{11}} = \text{-----}$



Key Observation - The 180° Rotation Pattern:

Examine your expression for $\frac{\partial L}{\partial x_{22}}$. Notice how the filter weights pair with gradients:

- $\frac{\partial L}{\partial o_{11}}$ multiplies with f_{22} (diagonally opposite positions)
- $\frac{\partial L}{\partial o_{12}}$ multiplies with f_{21} (horizontally flipped)
- $\frac{\partial L}{\partial o_{21}}$ multiplies with f_{12} (vertically flipped)
- $\frac{\partial L}{\partial o_{22}}$ multiplies with f_{11} (fully rotated by 180°)

The filter weights appear in REVERSE ORDER compared to the forward pass! This 180° rotation is the key to understanding how gradients flow backward through convolutions.

(e) [2 points] Synthesis; Completing the backward pass:

Based on the rotation pattern you observed, to compute all elements of $\frac{\partial L}{\partial \mathbf{X}}$ efficiently, we perform a "full convolution" between:

- A **padded version** of $\frac{\partial L}{\partial \mathbf{O}}$ (padded with zeros on all sides)
- The **180° rotated filter** $\text{rot180}(\mathbf{F}) = \begin{bmatrix} f_{22} & f_{21} \\ f_{12} & f_{11} \end{bmatrix}$

Answer these:

- What size should the padded gradient matrix be to produce a 3×3 output? (Hint: use the output size formula backward)
- Write out the 4×4 padded gradient matrix explicitly (pad $\frac{\partial L}{\partial \mathbf{O}}$ with zeros).

- (iii) Verify: Apply $\text{rot180}(\mathbf{F})$ at position (0,0) of your padded matrix. What does it produce?

What You Just Proved

You have rigorously shown that CNN backpropagation consists of two convolution operations:

1. $\frac{\partial L}{\partial \mathbf{F}} = \text{conv2d}(\mathbf{X}, \frac{\partial L}{\partial \mathbf{O}})$ — Filter gradients
2. $\frac{\partial L}{\partial \mathbf{X}} = \text{conv2d}_{\text{full}}(\frac{\partial L}{\partial \mathbf{O}}, \text{rot180}(\mathbf{F}))$ — Input gradients

Analytical Reflection [3 points]

Now that you've proven the mathematical equivalence, answer these conceptual questions:

- (a) [1 point] In the expression $\frac{\partial L}{\partial \mathbf{F}} = \text{conv2d}(\mathbf{X}, \frac{\partial L}{\partial \mathbf{O}})$, which matrix acts as the "filter" and which acts as the "input" for this backward convolution? How does this compare to the forward pass roles?

- (b) [1 point] Why must we rotate the filter by 180° when computing $\frac{\partial L}{\partial \mathbf{X}}$ but not when computing $\frac{\partial L}{\partial \mathbf{F}}$? (Hint: think about the direction of information flow and which variables are "parameters" vs "activations")

- (c) [1 point] In a deep CNN with many convolutional layers, gradients must flow backward through all layers. Based on your proof, you might see this backward gradient flow is computationally efficient compared to if we had to compute Jacobian matrices explicitly. What property of convolution makes this **efficiency** possible?

Impact: CNN backpropagation reuses the same optimized convolution operations as forward pass. This is why GPUs can train CNNs efficiently, the same hardware kernels serve both directions! Modern deep learning frameworks (PyTorch, TensorFlow) exploit this mathematical equivalence for automatic differentiation. Hope this helps illuminate why CNNs train so effectively in practice!

Extra page (1) (do not attach additional pages)

Use this page only if you need extra space for any of the above questions. Provide clear references to the question numbers and parts.

Extra page (2) (do not attach additional pages)

Use this page only if you need extra space for any of the above questions. Provide clear references to the question numbers and parts.